LAPORAN PROYEK RSA SECURE DIGITAL PROCUREMENT & TENDER MANAGEMENT SYSTEM



Disusun oleh Kelompok 5

Putu Yudi Nandanjaya Wiraguna	5027241080
Raihan Fahri Ghazali	5027241061
I Gede Bagus Saka Sinatrya	5027241088

MATA KULIAH KRIPTOGRAFI
DEPARTEMEN TEKNOLOGI INFORMASI
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

Abstrak

Pengadaan digital memerlukan perlindungan terhadap kerahasiaan informasi penawaran yang dipertukarkan antara penyedia dan panitia. Laporan ini menjelaskan perancangan dan implementasi laboratorium RSA pada TenderVault, sebuah aplikasi web satu halaman yang berfungsi sebagai prototipe demonstrasi kriptografi untuk skenario pengadaan dan tender digital. Ruang lingkup laporan dibatasi pada algoritma RSA: pembangkitan kunci dari dua bilangan prima, enkripsi pesan dengan kunci publik, dan dekripsi dengan kunci privat.

Aplikasi dibuat menggunakan HTML, CSS, dan JavaScript murni tanpa framework maupun library kriptografi. Seluruh fungsi inti RSA, yaitu gcd, extendedGcd, modInverse, modPow (perpangkatan modular dengan metode *square-and-multiply*), dan isPrime, ditulis sendiri pada app.js menggunakan aritmetika BigInt. Satu-satunya fasilitas bawaan browser yang bersifat kriptografis adalah crypto.getRandomValues, yang hanya dipakai untuk memilih bilangan prima acak dan tidak dipakai untuk menghitung RSA. Pesan dikodekan ke byte UTF-8, dan setiap byte dienkripsi secara terpisah dengan kunci publik (n, e) lalu dipulihkan dengan kunci privat (n, d).

Pengujian mencakup 12 skenario yang dijalankan secara otomatis pada fungsi internal dan antarmuka di browser Chromium, termasuk kasus masukan tidak valid dan ciphertext rusak. Seluruh skenario sesuai harapan pada versi akhir program; satu keterbatasan (pesan galat bawaan JavaScript untuk masukan notasi ilmiah atau desimal) dicatat secara terbuka. Selama pengujian ditemukan satu cacat pada versi awal berkas yang membuat tombol dekripsi selalu menolak masukan; cacat tersebut telah diperbaiki dan dijelaskan pada Bab 5. Aplikasi ini ditujukan untuk pembelajaran, bukan untuk melindungi data pengadaan nyata.

Identitas dan ruang lingkup

Bagian	Ruang lingkup implementasi
Studi kasus	Pembuatan pasangan kunci RSA serta enkripsi/dekripsi pesan teks UTF-8 untuk pertukaran informasi penawaran.
Antarmuka	Satu halaman laboratorium RSA yang berjalan di browser, dengan panel jejak perhitungan.
Posisi terhadap topik	Demonstrator kriptografi RSA untuk skenario tender, bukan sistem manajemen tender yang utuh.
Batasan	Demo akademik lokal; tidak ada autentikasi, penyimpanan tender, tanda tangan digital, distribusi kunci, atau protokol produksi.
Algoritma lain	Proyek induk TenderVault juga memuat laboratorium DES dan AES-128; bagian tersebut tidak termasuk dalam berkas dan laporan ini.

Daftar Isi

1. Pendahuluan..... 5
1.1 Latar belakang..... 5
1.2 Identifikasi masalah..... 5
1.3 Tujuan..... 5
1.4 Manfaat..... 5
1.5 Batasan masalah..... 6
2. Dasar Teori..... 7
2.1 Kriptografi asimetris..... 7
2.2 Algoritma RSA..... 7
2.3 Extended Euclidean Algorithm dan invers modular..... 7
2.4 Perpangkatan modular..... 7
2.5 Uji bilangan prima dengan pembagian percobaan..... 7
2.6 Pengodean pesan dan syarat modulus..... 8
2.7 Keamanan RSA textbook..... 8
3. Analisis dan Perancangan Sistem..... 9
3.1 Skenario pengadaan..... 9
3.2 Kebutuhan fungsional..... 9
3.3 Rancangan antarmuka..... 9
3.4 Alur dan status aplikasi..... 10
3.5 Struktur program..... 10
4. Implementasi Algoritma RSA..... 11
4.1 Fungsi matematika dasar..... 11
4.2 Pemilihan bilangan prima acak..... 12
4.3 Pembangkitan pasangan kunci..... 12
4.4 Enkripsi..... 12
4.5 Dekripsi..... 13
4.6 Contoh perhitungan dengan nilai bawaan..... 13
4.7 Contoh dengan prima acak..... 14
4.8 Keterbatasan implementasi..... 14
5. Pengujian dan Hasil..... 15
5.1 Metode pengujian..... 15
5.2 Hasil pengujian..... 15
5.3 Analisis hasil..... 16
6. Bukti Tampilan Program..... 17
7. Kesimpulan dan Saran..... 23
7.1 Kesimpulan..... 23
7.2 Saran pengembangan..... 23
Daftar Pustaka..... 24

Cara menjalankan program

Buka index.html pada browser modern; tidak diperlukan instalasi maupun server. Fitur pemilihan prima acak memerlukan browser yang menyediakan crypto.getRandomValues. Berkas program: index.html, styles.css, app.js, dan README.md.

1. Pendahuluan

1.1 Latar belakang

Proses pengadaan dan tender digital melibatkan pertukaran informasi yang sensitif, seperti rincian harga, spesifikasi, dan dokumen penawaran. Apabila informasi tersebut dapat dibaca pihak yang tidak berwenang, kerahasiaan dan kepercayaan dalam proses pengadaan dapat terganggu. Kriptografi menyediakan teknik untuk mengubah informasi yang dapat dibaca menjadi ciphertext dan mengembalikannya melalui kunci yang sesuai [5].

Pada kriptografi simetris, pengirim dan penerima harus lebih dahulu berbagi satu kunci rahasia, padahal penyedia dan panitia pengadaan sering belum saling mengenal. RSA [1] mengatasi masalah ini dengan pasangan kunci: kunci publik boleh disebarkan kepada siapa pun dan dipakai untuk enkripsi, sedangkan kunci privat dirahasiakan oleh pemiliknya dan dipakai untuk dekripsi. Dengan demikian penyedia dapat mengenkripsi informasi penawaran menggunakan kunci publik panitia tanpa pertukaran rahasia sebelumnya.

Sesuai ketentuan tugas, RSA diimplementasikan tanpa library kriptografi dan harus memperlihatkan tahap pembangkitan kunci, enkripsi, dan dekripsi. TenderVault memvisualisasikan ketiga tahap tersebut melalui antarmuka yang dapat dicoba langsung. Proyek induk TenderVault juga memuat laboratorium DES dan AES-128; laporan ini hanya membahas bagian RSA sesuai ruang lingkup demonstrasi, sehingga berkas yang diserahkan hanya memuat laboratorium RSA.

1.2 Identifikasi masalah

1. Bagaimana membangkitkan pasangan kunci RSA dari dua bilangan prima tanpa library kriptografi, dan bagaimana menunjukkan hubungan antara kunci publik dan kunci privat?
2. Bagaimana mengenkripsi dan mendekripsi pesan teks (termasuk karakter non-ASCII) dengan RSA, sehingga pesan dapat dipulihkan tepat seperti aslinya?
3. Bagaimana menyediakan antarmuka yang menampilkan tahapan perhitungan dan menolak masukan yang tidak valid dengan pesan yang jelas?

1.3 Tujuan

- Membangun aplikasi demonstrasi RSA berbasis antarmuka web untuk topik pengadaan digital.
- Menerapkan RSA secara mandiri dengan BigInt, dari pengujian bilangan prima hingga pemulihan pesan.
- Menampilkan jejak perhitungan n, $\varphi(n)$, e, dan d agar proses dapat diikuti pengguna.
- Menguji hasil melalui uji round-trip, verifikasi perhitungan manual, dan skenario masukan tidak valid.

1.4 Manfaat

Program membantu pengguna memahami fungsi setiap nilai dalam RSA, perbedaan peran kunci publik dan kunci privat, serta alasan RSA *textbook* dengan bilangan kecil tidak aman. Demonstrasi juga memperjelas bahwa ciphertext berupa deretan bilangan yang tidak terbaca sebagai teks biasa dan bahwa dekripsi memerlukan kunci privat yang tepat.

1.5 Batasan masalah

TenderVault adalah demonstrator kriptografi untuk skenario tender, bukan sistem manajemen tender yang lengkap. Masukan berupa teks pada kotak pesan, bukan unggahan berkas tender. Aplikasi tidak menyimpan akun, dokumen, atau kunci; tidak melakukan komunikasi jaringan; dan tidak menyediakan autentikasi, tanda tangan digital, distribusi kunci, ataupun pengelolaan siklus hidup kunci. Bilangan prima dibatasi pada ukuran kecil sehingga aplikasi hanya prototipe pendidikan.

2. Dasar Teori

2.1 Kriptografi asimetris

Kriptografi asimetris menggunakan sepasang kunci yang berhubungan secara matematis: kunci publik dan kunci privat. Pesan yang dienkripsi dengan kunci publik hanya dapat dibuka dengan kunci privat pasangannya. Keamanannya bertumpu pada masalah yang mudah dihitung ke satu arah tetapi sulit dibalik; pada RSA, masalah tersebut adalah memfaktorkan hasil kali dua bilangan prima besar [1].

2.2 Algoritma RSA

RSA terdiri atas tiga tahap. Pada pembangkitan kunci dipilih dua bilangan prima berbeda p dan q, lalu dihitung modulus n dan fungsi totien Euler $\varphi(n)$. Eksponen publik e dipilih sehingga $1 < e < \varphi(n)$ dan relatif prima terhadap $\varphi(n)$, sedangkan eksponen privat d adalah invers modular dari e.

$$n = p \times q \qquad \varphi(n) = (p-1)(q-1)$$
$$\gcd(e, \varphi(n)) = 1 \qquad e \times d \equiv 1 \pmod{\varphi(n)}$$

Kunci publik adalah pasangan (n, e) dan kunci privat adalah (n, d). Untuk pesan berupa bilangan m dengan $0 \le m < n$, enkripsi dan dekripsi dilakukan sebagai berikut.

$$\text{Enkripsi: } c = m^e \bmod n \qquad \text{Dekripsi: } m = c^d \bmod n$$

Kebenaran dekripsi mengikuti teorema Euler. Karena $e \times d = 1 + k \cdot \varphi(n)$ untuk suatu bilangan bulat k, maka $c^d = m^{ed} = m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}$ untuk m yang relatif prima terhadap n. Karena n adalah hasil kali dua prima berbeda, kesamaan ini juga berlaku untuk seluruh m pada rentang 0 hingga $n - 1$ [1].

2.3 Extended Euclidean Algorithm dan invers modular

Algoritma Euclid menghitung $\gcd(a, b)$ melalui pembagian berulang. Versi yang diperluas (*extended*) sekaligus menghasilkan bilangan bulat x dan y yang memenuhi $a \cdot x + b \cdot y = \gcd(a, b)$. Jika $\gcd(e, \varphi(n)) = 1$, nilai x tersebut adalah invers e modulo $\varphi(n)$, sehingga $d \equiv x \pmod{\varphi(n)}$. Ketiadaan invers ($\gcd \neq 1$) menandakan e tidak dapat dipakai.

2.4 Perpangkatan modular

Menghitung m^e secara langsung lalu mengambil sisa pembagian menghasilkan bilangan yang sangat besar. Metode *square-and-multiply* memeriksa bit-bit eksponen dari yang terendah: basis dikuadratkan pada setiap langkah dan dikalikan ke hasil bila bit eksponen bernilai 1, dengan sisa pembagian modulo n diambil di setiap langkah. Jumlah perkalian sebanding dengan $\log e$, bukan e.

2.5 Uji bilangan prima dengan pembagian percobaan

Bilangan v adalah prima bila tidak habis dibagi bilangan ganjil mana pun dari 3 sampai $\sqrt{v}$ (setelah memeriksa bilangan 2). Metode ini sederhana dan benar, dengan jumlah percobaan sekitar $\sqrt{v}/2$, sehingga sesuai untuk prima kecil tetapi tidak praktis untuk bilangan besar.

2.6 Pengodean pesan dan syarat modulus

RSA bekerja pada bilangan, bukan teks. Pada demo, pesan dikodekan menjadi byte UTF-8 dan setiap byte (nilai 0–255) diperlakukan sebagai satu bilangan m. Agar $m < n$ untuk semua kemungkinan byte, modulus harus memenuhi $n > 255$.

2.7 Keamanan RSA textbook

RSA yang dipakai pada demo adalah RSA *textbook*: enkripsi langsung tanpa padding acak. Skema ini bersifat deterministik sehingga byte yang sama selalu menghasilkan ciphertext yang sama, dan pola pesan dapat dianalisis dengan frekuensi. RSA produksi memerlukan padding seperti OAEP serta ukuran kunci yang kuat [2], [3], dan lazimnya dipakai untuk melindungi kunci sesi simetris, bukan dokumen besar. Prima kecil seperti pada demo dapat difaktorkan dengan mudah, sehingga kunci privat dapat dihitung ulang oleh pihak lain (lihat Bagian 4.8).

3. Analisis dan Perancangan Sistem

3.1 Skenario pengadaan

Panitia pengadaan memiliki pasangan kunci RSA. Kunci publik dapat dibagikan kepada penyedia, sedangkan kunci privat tetap dirahasiakan oleh panitia. Penyedia menyusun informasi penawaran, misalnya nilai harga, lalu mengenkripsinya dengan kunci publik sehingga hanya panitia yang dapat membacanya kembali dengan kunci privat.

Pada demo, kedua peran dijalankan oleh satu halaman web: bagian *Pembangkitan kunci* mewakili panitia yang membuat pasangan kunci, dan bagian *Enkripsi & dekripsi pesan* mewakili penyedia (enkripsi) dan panitia (dekripsi). Pengiriman kunci publik dan ciphertext antarpihak tidak dimodelkan; hal ini dicatat sebagai batasan.

3.2 Kebutuhan fungsional

Kode	Kebutuhan
F1	Menerima bilangan prima p dan q dari pengguna, atau memilihnya secara acak.
F2	Memvalidasi masukan: p dan q berbeda, keduanya prima, dan $n > 255$.
F3	Menghitung dan menampilkan n, $\varphi(n)$, e, dan d beserta jejak rumusnya.
F4	Mengenkripsi pesan UTF-8 per byte dengan kunci publik dan menampilkan ciphertext berupa daftar bilangan.
F5	Mendekripsi ciphertext dengan kunci privat, memvalidasi format dan rentang, lalu menampilkan teks hasil pemulihan.
F6	Membatalkan kunci dan menonaktifkan tombol bila p atau q diubah setelah kunci dibuat.

3.3 Rancangan antarmuka

Antarmuka terdiri atas satu halaman dengan dua kartu kerja di kolom kiri dan panel penjelasan di kolom kanan.

Panel	Konten	Interaksi
01 Pembangkitan kunci	Kolom prima P dan Q, rumus pembangkitan, kotak kunci publik (n, e) dan kunci privat (n, d), pesan galat.	Tombol *Acak prima* dan *Bangkitkan pasangan kunci*.
02 Enkripsi & dekripsi pesan	Kotak pesan tender, kotak ciphertext (integer), baris status.	Tombol *Enkripsi dengan kunci publik* dan *Dekripsi*.
Jejak perhitungan	Lima langkah: modulus, totien Euler, eksponen publik, eksponen privat, operasi pesan, dengan nilai aktual.	Diperbarui otomatis saat kunci dibuat.
Catatan keamanan	Peringatan bahwa RSA kecil hanya untuk belajar, dan penjelasan siapa yang dapat membuka pesan.	Informasi statis.

3.4 Alur dan status aplikasi

Aplikasi menyimpan satu variabel status rsaKeys yang bernilai kosong (*null*) atau berisi {p, q, n, φ, e, d}. Tombol enkripsi aktif hanya bila rsaKeys terisi; tombol dekripsi aktif hanya bila rsaKeys terisi dan kotak ciphertext tidak kosong. Perubahan nilai p atau q memanggil invalidateRsaKeys() yang mengosongkan kunci, ciphertext, dan seluruh tampilan nilai agar kunci lama tidak terpakai secara tidak sengaja. Saat halaman dibuka, kunci awal dibangkitkan otomatis dari nilai bawaan $p = 61$ dan $q = 53$.

3.5 Struktur program

Berkas	Peran
index.html	Struktur halaman, kolom masukan, tombol, panel hasil, dan peringatan keamanan.
styles.css	Tata letak responsif, warna, komponen kartu, navigasi, dan tampilan laboratorium.
app.js	Implementasi RSA (fungsi matematika, pembangkitan kunci, enkripsi, dekripsi), validasi masukan, dan pengendali interaksi.
README.md	Petunjuk menjalankan aplikasi serta catatan penggunaan dan keamanan.

4. Implementasi Algoritma RSA

Seluruh logika RSA berada pada app.js dan memakai tipe BigInt agar perhitungan bilangan bulat tidak kehilangan ketepatan. Tidak ada library eksternal yang dipakai.

4.1 Fungsi matematika dasar

Pembagi persekutuan terbesar. Fungsi gcd memakai algoritma Euclid iteratif, dan extendedGcd memakai versi rekursif yang mengembalikan tiga nilai [g, x, y] sehingga $a \cdot x + b \cdot y = g$.

```
function gcd(a, b) {
  while (b !== 0n) [a, b] = [b, a % b];
  return a < 0n ? -a : a;
}

function extendedGcd(a, b) {
  if (b === 0n) return [a, 1n, 0n];
  const [g, x, y] = extendedGcd(b, a % b);
  return [g, y, x - (a / b) * y];
}
```

Invers modular. modInverse memanggil extendedGcd; jika gcd tidak sama dengan 1 maka dilempar galat, dan hasil dinormalkan ke rentang 0 hingga modulus − 1.

```
function modInverse(value, modulus) {
  const [g, x] = extendedGcd(value, modulus);
  if (g !== 1n) throw new Error("Eksponen publik tidak memiliki invers modulo φ(n).");
  return ((x % modulus) + modulus) % modulus;
}
```

Perpangkatan modular. modPow menerapkan *square-and-multiply*: pada setiap iterasi bit terendah eksponen diperiksa (exponent & 1n), basis dikuadratkan, dan eksponen digeser satu bit.

```
function modPow(base, exponent, modulus) {
  let result = 1n;
  base %= modulus;
  while (exponent > 0n) {
    if (exponent & 1n) result = (result * base) % modulus;
    exponent >>= 1n;
    base = (base * base) % modulus;
  }
  return result;
}
```

Uji prima. isPrime menolak bilangan kurang dari 2 dan bilangan genap selain 2, lalu mencoba pembagi ganjil dari 3 hingga $\sqrt{v}$.

```
function isPrime(value) {
  if (value < 2n) return false;
  if (value === 2n) return true;
  if (value % 2n === 0n) return false;
  for (let divisor = 3n; divisor * divisor <= value; divisor += 2n) {
    if (value % divisor === 0n) return false;
  }
  return true;
}
```

4.2 Pemilihan bilangan prima acak

Tombol *Acak prima* memanggil randomPrime, yang mengambil bilangan acak pada rentang 257–997, menjadikannya ganjil, lalu mengujinya dengan isPrime hingga ditemukan prima (maksimal 10.000 percobaan). Nilai acak berasal dari crypto.getRandomValues; fasilitas ini hanya dipakai untuk memilih p dan q, bukan untuk menghitung RSA. Pengacak memastikan q tidak sama dengan p. Karena prima terkecil pada rentang tersebut adalah 257, nilai n selalu jauh di atas 255.

4.3 Pembangkitan pasangan kunci

Fungsi generateRsaKeys membaca p dan q dari kolom masukan, memvalidasi, lalu menghitung kunci. Setiap kegagalan menghasilkan pesan galat di bawah tombol dan memanggil invalidateRsaKeys() sehingga tidak ada kunci setengah jadi yang tersisa.

Urut	Proses	Keterangan
1	Cek $p \neq q$	Bila sama: “Bilangan prima p dan q harus berbeda.”
2	Cek keprimaan	Bila salah satu bukan prima: “Nilai p dan q harus berupa bilangan prima.”
3	Hitung $n = p \times q$	Bila $n \leq 255$: “Hasil n harus lebih besar dari 255 agar setiap byte pesan dapat dienkripsi.”
4	Hitung $\varphi(n) = (p-1)(q-1)$	Totien Euler untuk hasil kali dua prima.
5	Pilih e	Coba $e = 65537$ bila $e < \varphi(n)$ dan $\gcd(e, \varphi(n)) = 1$; jika tidak, cari bilangan ganjil mulai dari 3 yang relatif prima terhadap $\varphi(n)$. Bila tidak ada, tampilkan galat.
6	Hitung d	modInverse(e, φ), yaitu d sehingga $e \cdot d \equiv 1 \pmod{\varphi(n)}$.
7	Tampilkan hasil	Mengisi kotak kunci publik (n, e), kunci privat (n, d), dan jejak perhitungan; mengaktifkan tombol enkripsi.

```
const n = p * q;
if (n <= 255n) throw new Error("Hasil n harus lebih besar dari 255 ...");
const phi = (p - 1n) * (q - 1n);
let e = 65537n;
if (e >= phi || gcd(e, phi) !== 1n) {
  e = 3n;
  while (e < phi && gcd(e, phi) !== 1n) e += 2n;
}
if (e >= phi) throw new Error("Tidak dapat memilih eksponen publik ...");
const d = modInverse(e, phi);
rsaKeys = { p, q, n, phi, e, d };
```

4.4 Enkripsi

Fungsi encryptRsaMessage mengodekan pesan ke byte UTF-8 dengan TextEncoder, menolak pesan kosong, lalu menghitung $c = m^e \bmod n$ untuk setiap byte dengan modPow. Hasilnya berupa bilangan desimal yang dipisahkan spasi. Satu byte pesan menjadi satu bilangan ciphertext, sehingga karakter non-ASCII (yang memakai 2 hingga 4 byte pada UTF-8) menghasilkan beberapa bilangan.

```
const bytes = encoder.encode($("#rsa-message").value);
const numbers = [];
for (const byte of bytes)
  numbers.push(modPow(BigInt(byte), rsaKeys.e, rsaKeys.n).toString());
$("#rsa-ciphertext").value = numbers.join(" ");
```

4.5 Dekripsi

Fungsi decryptRsaMessage memecah ciphertext berdasarkan spasi lalu memvalidasi setiap nilai sebelum diproses: (1) seluruh token harus berupa angka desimal tanpa tanda, (2) setiap nilai harus berada pada rentang 0 hingga $n - 1$, dan (3) hasil dekripsi harus berada pada rentang byte (0–255). Setelah lolos, byte digabung dan didekode dengan TextDecoder mode ketat (fatal: true) sehingga urutan UTF-8 yang tidak valid memunculkan galat, bukan karakter pengganti.

```
const entered = $("#rsa-ciphertext").value.trim().split(/\s+/).filter(Boolean);
if (!entered.length || entered.some((v) => !/^\d+$/.test(v)))
  throw new Error("Ciphertext harus berupa deretan integer desimal.");
const bytes = entered.map((value) => {
  const cipher = BigInt(value);
  if (cipher < 0n || cipher >= rsaKeys.n) throw new Error("... rentang 0 hingga n−1.");
  const plain = modPow(cipher, rsaKeys.d, rsaKeys.n);
  if (plain > 255n) throw new Error("Hasil dekripsi bukan byte yang valid ...");
  return Number(plain);
});
const message = decoder.decode(Uint8Array.from(bytes));
```

4.6 Contoh perhitungan dengan nilai bawaan

Tabel berikut menunjukkan kunci yang dibangkitkan dari nilai bawaan pada antarmuka.

Parameter	Perhitungan	Nilai
Prima *p* dan *q*	Masukan bawaan pada UI	$p = 61$, $q = 53$
Modulus *n*	61 × 53	3233
Totien φ(*n*)	60 × 52	3120
Eksponen publik *e*	65537 > φ(*n*), sehingga dicari bilangan ganjil kecil: gcd(3, 3120) = 3 dan gcd(5, 3120) = 5 (ditolak), gcd(7, 3120) = 1 (dipilih)	7
Eksponen privat *d*	Invers 7 modulo 3120; 7 × 1783 = 12481 = 4 × 3120 + 1	1783
Kunci publik / privat	(*n*, *e*) / (*n*, *d*)	(3233, 7) / (3233, 1783)

Penelusuran extended Euclid untuk φ(*n*) = 3120 dan *e* = 7:

```
3120 = 445 × 7 + 5
   7 =   1 × 5 + 2
   5 =   2 × 2 + 1
Substitusi balik:
   1 = 5 − 2×2 = 5 − 2×(7 − 5) = 3×5 − 2×7
     = 3×(3120 − 445×7) − 2×7 = 3×3120 − 1337×7
=> 7⁻¹ ≡ −1337 ≡ 3120 − 1337 = 1783 (mod 3120)
```

Penelusuran square-and-multiply untuk huruf “a” (kode 97), eksponen 7 = 111_2, modulus 3233:

Langkah	Basis (setelah kuadrat)	Hasil sementara
Bit 1 (1)	97^2 mod 3233 = 2943	hasil = 1 × 97 = 97
Bit 2 (1)	2943^2 mod 3233 = 42	hasil = 97 × 2943 mod 3233 = 967
Bit 3 (1)	42^2 mod 3233 = 1764	hasil = 967 × 42 mod 3233 = 1818

Jadi 97^7 mod 3233 = **1818**. Dekripsi: 1818^{1783} mod 3233 = **97**, yaitu kembali menjadi huruf “a”. Untuk kata “Harga”, byte 72, 97, 114, 103, 97 menjadi ciphertext 1087, 1818, 1797, 728, 1818. Nilai 1818 muncul dua kali karena huruf “a” yang sama selalu menghasilkan bilangan yang sama.

4.7 Contoh dengan prima acak

Pada salah satu pengacakan (Gambar 4), diperoleh *p* = 409 dan *q* = 269 sehingga *n* = 110021, φ(*n*) = 408 × 268 = 109344, dan *e* = 65537 karena 65537 < φ(*n*) dan relatif prima terhadap φ(*n*). Eksponen privat yang dihitung adalah *d* = 30881, dan 65537 × 30881 mod 109344 = 1. Karena *p* dan *q* dipilih acak, nilai ini akan berbeda pada setiap pengacakan.

4.8 Keterbatasan implementasi

- **Deterministik.** Byte yang sama selalu menghasilkan ciphertext yang sama (pada kunci bawaan, huruf “a” selalu menjadi 1818 dan angka “0” selalu menjadi 2488), sehingga pola pesan dapat dianalisis dengan frekuensi. Ini sifat RSA *textbook* tanpa padding. Byte bernilai 0 dan 1 bahkan tidak berubah oleh enkripsi.
- **Prima kecil mudah difaktorkan.** Modulus 3233 dapat difaktorkan menjadi 53 × 61 hanya dengan 26 percobaan pembagian ganjil, dan 110021 menjadi 269 × 409 dengan 134 percobaan. Setelah *p* dan *q* diketahui, φ(*n*) dan *d* dapat dihitung dengan rumus yang sama seperti pada aplikasi.
- **Uji prima sederhana.** Pembagian percobaan cukup untuk prima kecil, tetapi lambat untuk bilangan besar; memasukkan bilangan sangat besar secara manual dapat memperlambat browser.
- **Format masukan p dan q.** Nilai dalam notasi ilmiah (misalnya 1e5) atau desimal (misalnya 6.5) ditolak, tetapi pesan galatnya berasal dari JavaScript (“Cannot convert 1e5 to a BigInt”), bukan pesan khusus aplikasi (skenario U12).

- **Tanpa integritas dan autentikasi.** Ciphertext tidak dilindungi pemeriksaan integritas dan tidak ada tanda tangan digital, sehingga penerima tidak dapat memastikan siapa pengirimnya.
- **Distribusi kunci tidak dimodelkan.** Kunci publik, kunci privat, dan ciphertext berada pada halaman yang sama; mendekripsi ciphertext lama memerlukan p dan q yang sama untuk membangkitkan ulang pasangan kunci.

5. Pengujian dan Hasil

5.1 Metode pengujian

Pengujian dilakukan pada dua tingkat. Pertama, fungsi internal app.js (pembangkitan kunci, enkripsi, dekripsi, dan validasi) dijalankan melalui skrip Node.js dengan DOM tiruan untuk memeriksa hasil numerik dan pesan galat. Kedua, antarmuka pada index.html dijalankan di browser Chromium dengan skrip otomatis Playwright yang mengikuti alur pengguna: mengisi kolom, menekan tombol, dan membaca keluaran. Tangkapan layar pada Lampiran A diambil dari eksekusi otomatis tersebut. Nilai kunci pada skenario U01 dan U02 juga dicocokkan dengan perhitungan manual pada Bagian 4.6.

5.2 Hasil pengujian

ID	Skenario	Hasil yang diharapkan	Hasil
U01	RSA bawaan $p = 61$, $q = 53$; pesan “Harga penawaran: Rp 250.000.000”.	$n = 3233$, $\varphi(n) = 3120$, $e = 7$, $d = 1783$; 31 byte terenkripsi dan pulih tepat.	Lulus (Gambar 1–3).
U02	Verifikasi manual satu byte: huruf “a” (97) dengan kunci bawaan.	$97^7 \bmod 3233 = 1818$ dan $1818^{1783} \bmod 3233 = 97$.	Lulus; cocok dengan Bagian 4.6.
U03	Prima acak (contoh $p = 409$, $q = 269$) dan pesan Unicode “Tender ü — 日本 ✓ Rp 1.000”.	Kunci terbentuk ($n = 110021$, $e = 65537$, $d = 30881$); pesan pulih tepat termasuk karakter non-ASCII.	Lulus (Gambar 4).
U04	Tombol *Acak prima* diulang 30 kali, masing-masing dengan enkripsi lalu dekripsi pesan Unicode.	Pesan pulih tepat pada setiap pengulangan.	Lulus (30 dari 30).
U05	$p = q = 61$.	Ditolak: “Bilangan prima p dan q harus berbeda.” Kunci tidak dibuat.	Lulus.
U06	$p = 60$ (bukan prima), $q = 53$.	Ditolak: “Nilai p dan q harus berupa bilangan prima.” Tombol enkripsi/dekripsi nonaktif.	Lulus (Gambar 5).
U07	n tidak melebihi 255: $(p, q) = (5, 7)$ dan $(2, 127)$, yaitu $n = 35$ dan 254.	Ditolak: n harus lebih besar dari 255 agar setiap byte dapat dienkripsi.	Lulus.
U08	Ciphertext di luar rentang: “9999 12 5” ($n = 3233$).	Ditolak: nilai harus berada pada rentang 0 hingga $n-1$.	Lulus (Gambar 6).
U09	Format ciphertext salah: “abc 12”, “-5 3”, dan “1e5”.	Ditolak: ciphertext harus berupa deretan integer desimal.	Lulus.
U10	p diubah setelah kunci dan ciphertext dibuat.	Kunci lama dihapus, ciphertext dikosongkan, tombol enkripsi/dekripsi nonaktif, status meminta kunci dibangkitkan ulang.	Lulus.
U11	Pesan kosong saat enkripsi; kotak ciphertext dikosongkan.	Pesan kosong ditolak dengan “Masukkan pesan tender yang akan dienkripsi.”; tombol dekripsi nonaktif.	Lulus.

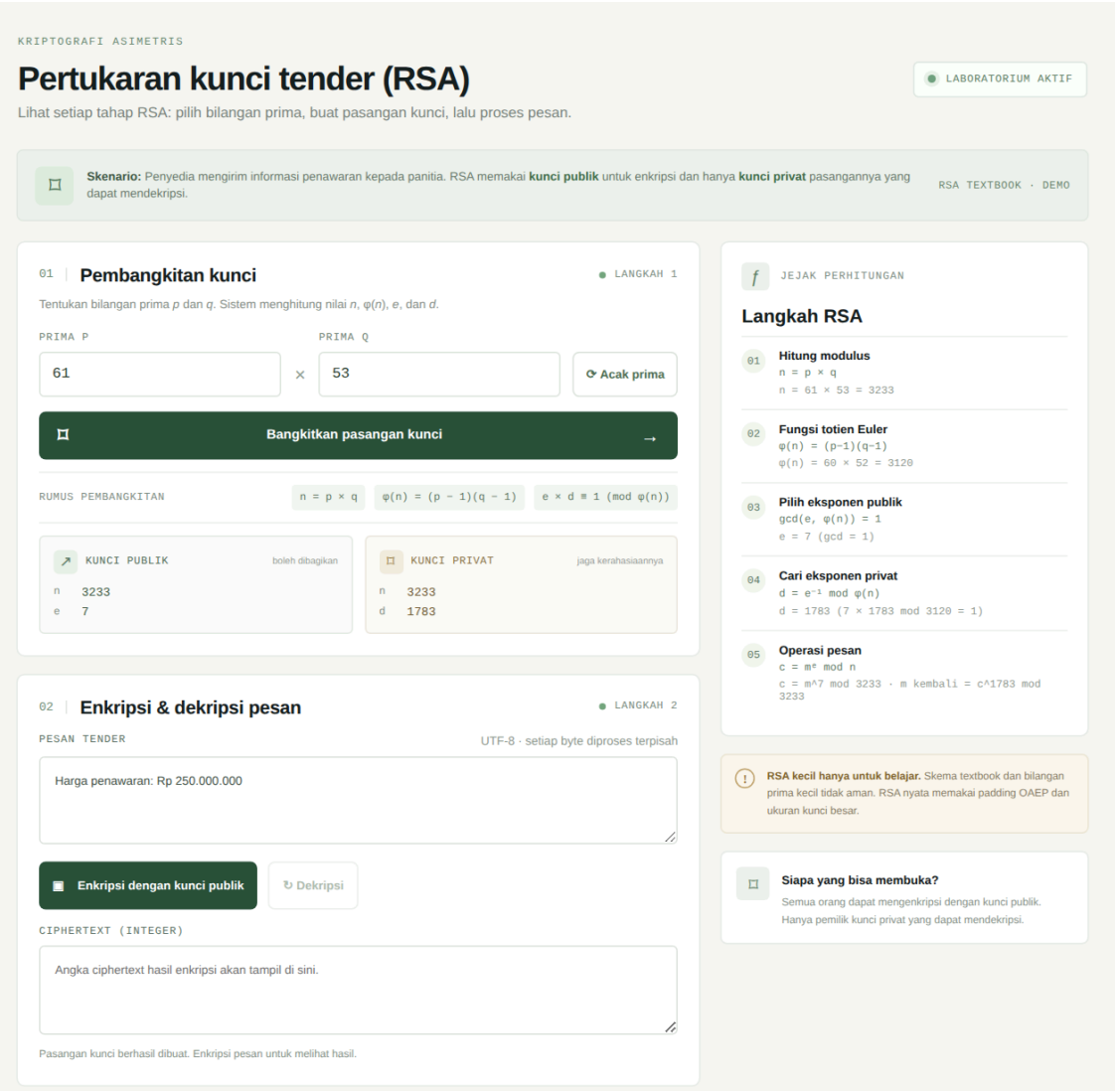
ID	Skenario	Hasil yang diharapkan	Hasil
U12	Masukan *p* dalam notasi ilmiah “1e5” dan desimal “6.5”.	Ditolak dengan pesan galat yang jelas dari aplikasi.	Ditolak, tetapi pesan bawaan JavaScript; dicatat sebagai keterbatasan (Bagian 4.8).

5.3 Analisis hasil

Skenario U01 hingga U04 menunjukkan bahwa seluruh rantai RSA (pemilihan prima, perhitungan n, $\varphi(n)$, e, d, enkripsi per byte, dan dekripsi) bekerja dan mengembalikan pesan secara tepat, termasuk pesan Unicode yang membutuhkan beberapa byte per karakter. Kecocokan kunci bawaan ($e = 7$, $d = 1783$) dengan perhitungan manual serta kecocokan nilai $97 \rightarrow 1818 \rightarrow 97$ menguatkan ketepatan modPow dan modInverse pada kasus yang dapat diverifikasi tangan. Skenario U05 hingga U11 menunjukkan bahwa masukan tidak valid dan ciphertext rusak ditolak dengan pesan yang sesuai tanpa menghentikan aplikasi, dan bahwa perubahan parameter membatalkan kunci lama.

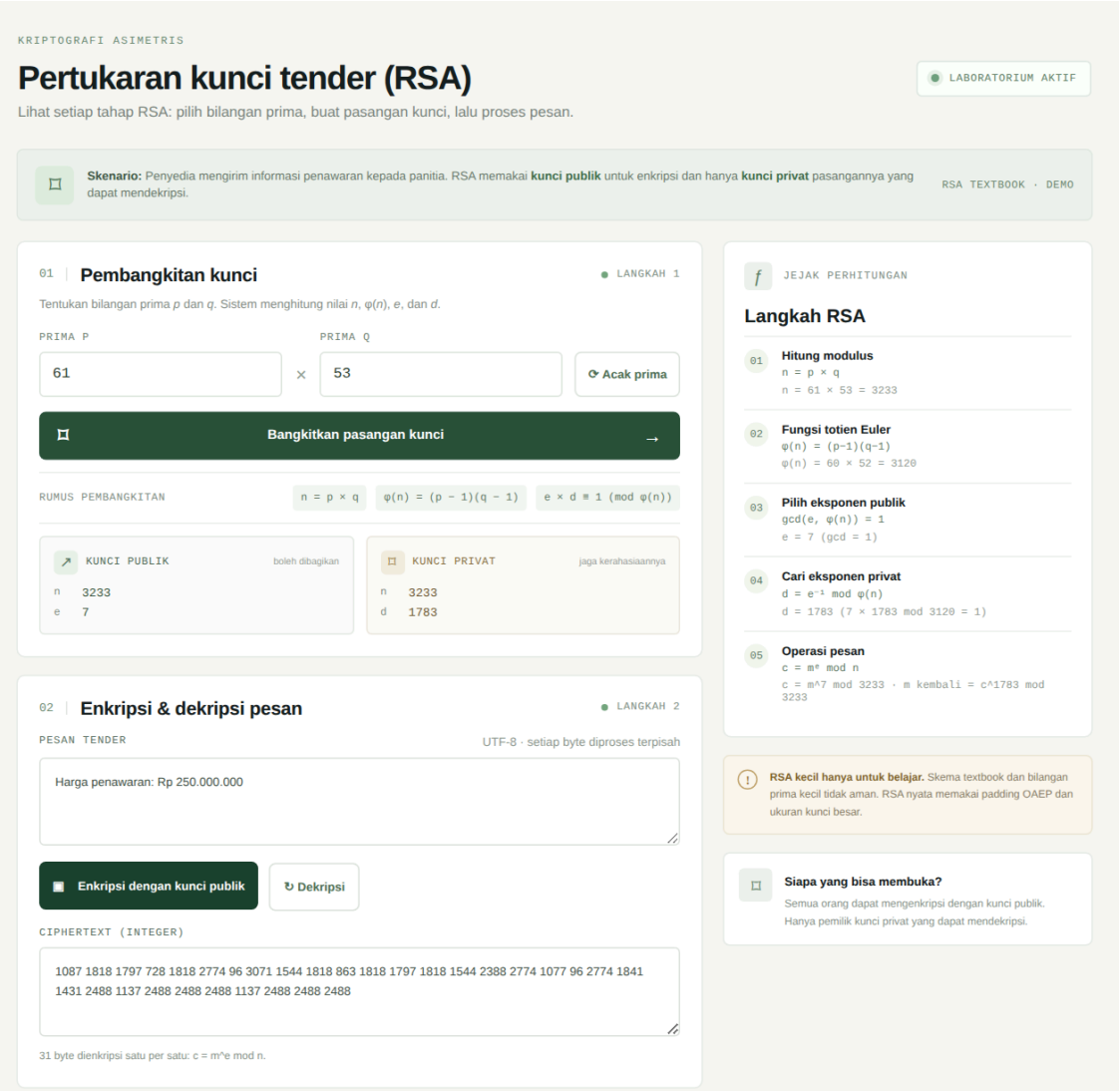
6. Bukti Tampilan Program

Lampiran A. Seluruh tangkapan layar diambil dari index.html pada Chromium secara otomatis, dengan data demonstrasi bawaan aplikasi dan bukan data penawaran sungguhan. Bagian bilah samping dan kaki halaman dipotong agar rincian terlihat lebih jelas.



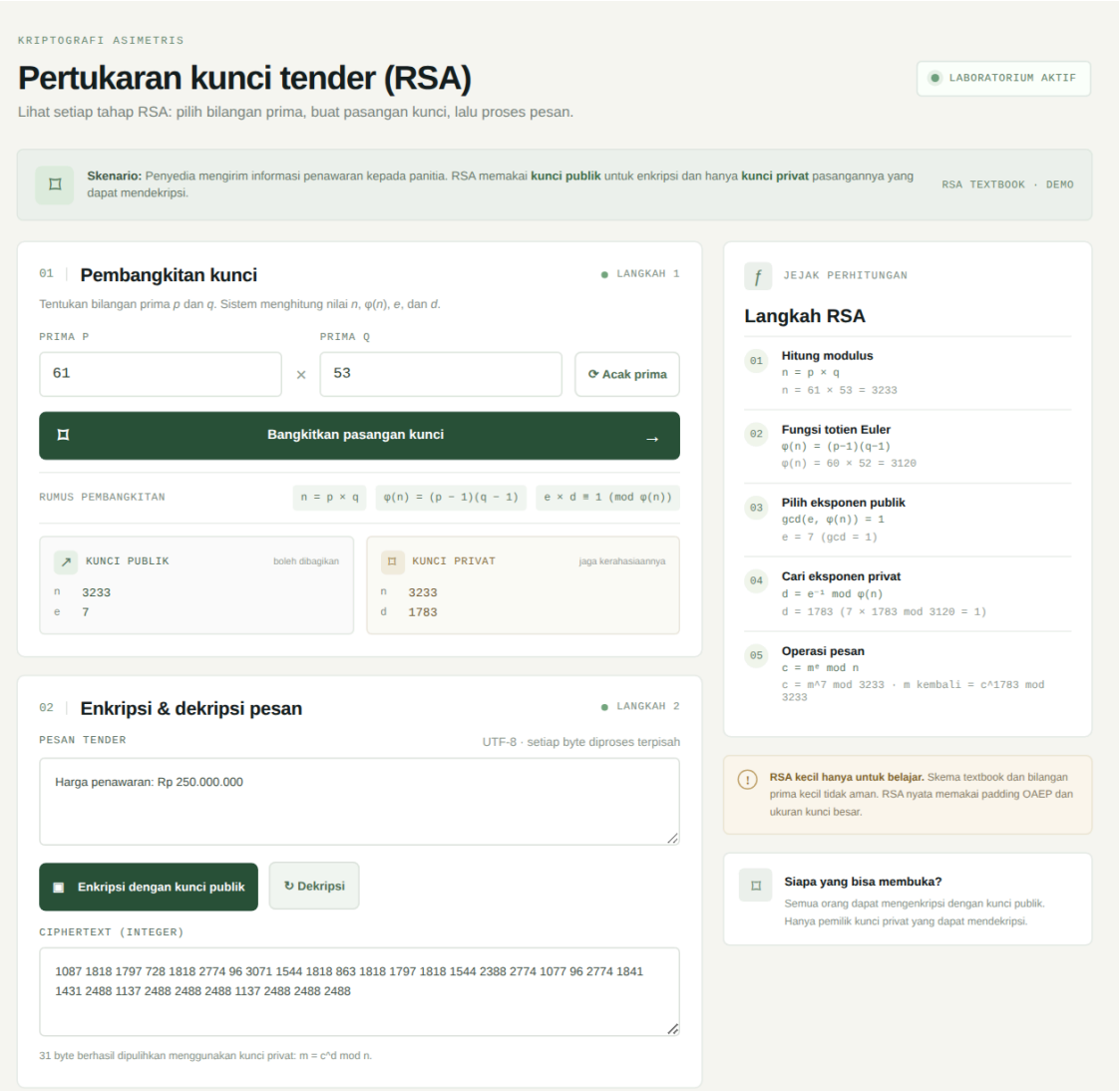
Gambar 1. Laboratorium RSA saat dibuka dengan nilai bawaan p = 61 dan q = 53: kunci publik (3233, 7), kunci privat (3233, 1783), dan jejak perhitungan.

Pada saat halaman dibuka, aplikasi langsung membangkitkan kunci dari nilai bawaan. Kotak kunci publik menampilkan (n, e) = (3233, 7) dan kotak kunci privat menampilkan (n, d) = (3233, 1783); panel kanan memperlihatkan tiap langkah perhitungan dengan nilai aktual.



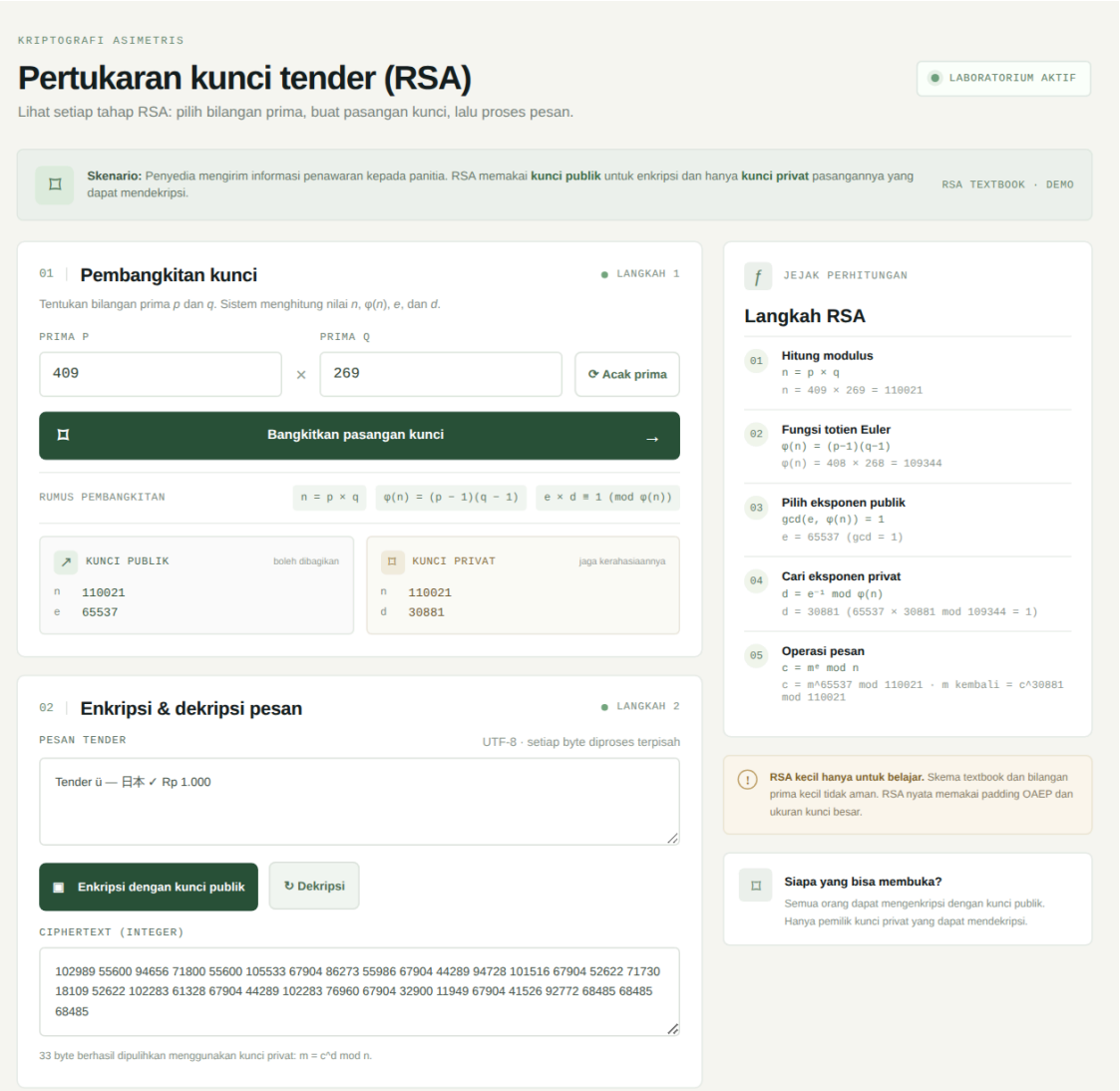
Gambar 2. Hasil enkripsi pesan "Harga penawaran: Rp 250.000.000" menjadi 31 bilangan ciphertext (satu per byte).

Pesan tender dikodekan ke 31 byte UTF-8, dan setiap byte dienkripsi dengan kunci publik menjadi satu bilangan. Terlihat bahwa huruf yang sama menghasilkan bilangan yang sama (misalnya 1818 untuk huruf "a"), sebagai ciri RSA textbook yang deterministik.



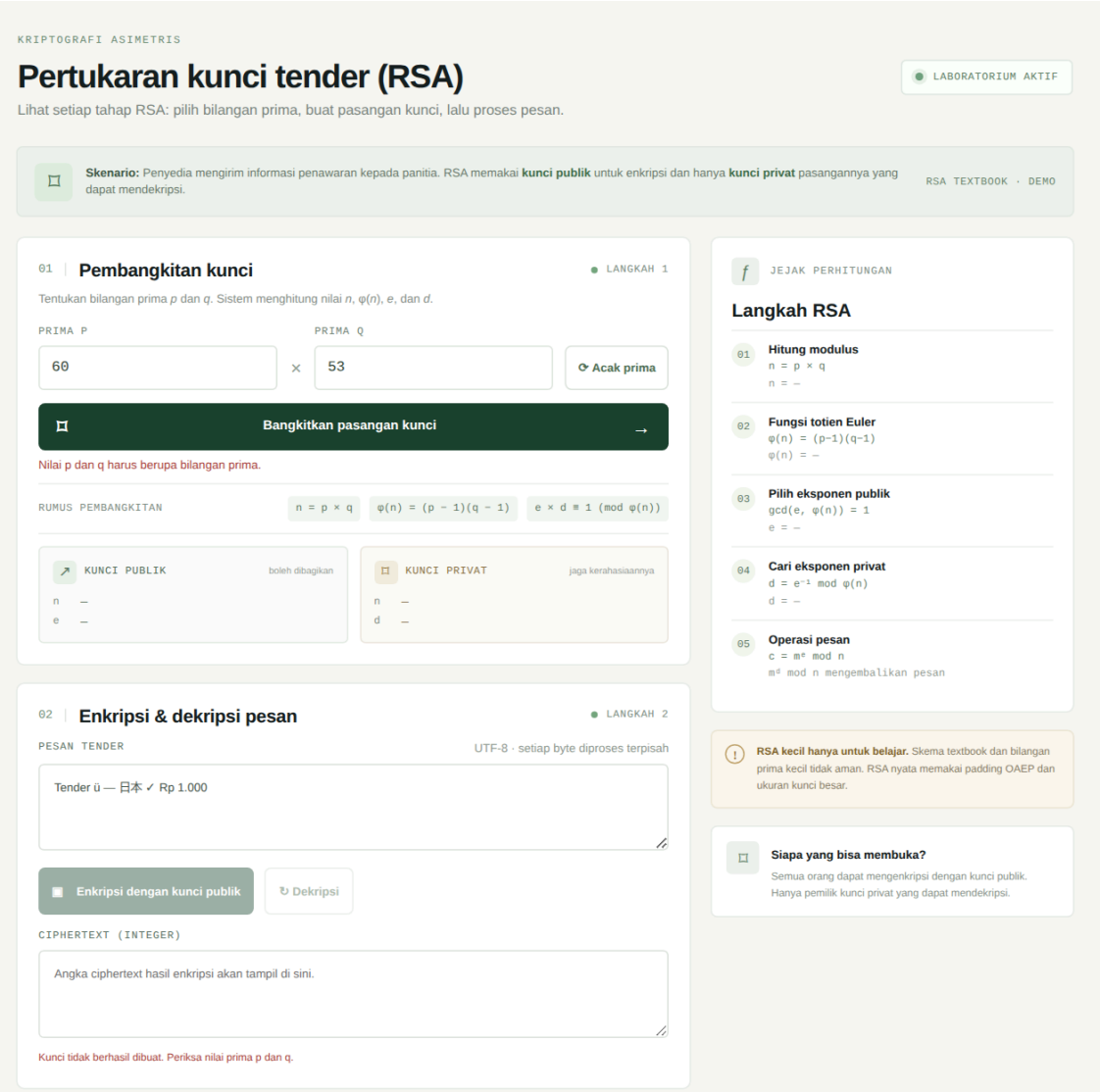
Gambar 3. Dekripsi ciphertext pada Gambar 2: kotak pesan dikosongkan sebelum dekripsi sehingga teks yang tampil adalah hasil pemulihan dengan kunci privat.

Setelah kotak pesan dikosongkan, tombol Dekripsi mengembalikan teks asli dari ciphertext dengan kunci privat. Baris status di bawah ciphertext melaporkan 31 byte berhasil dipulihkan.



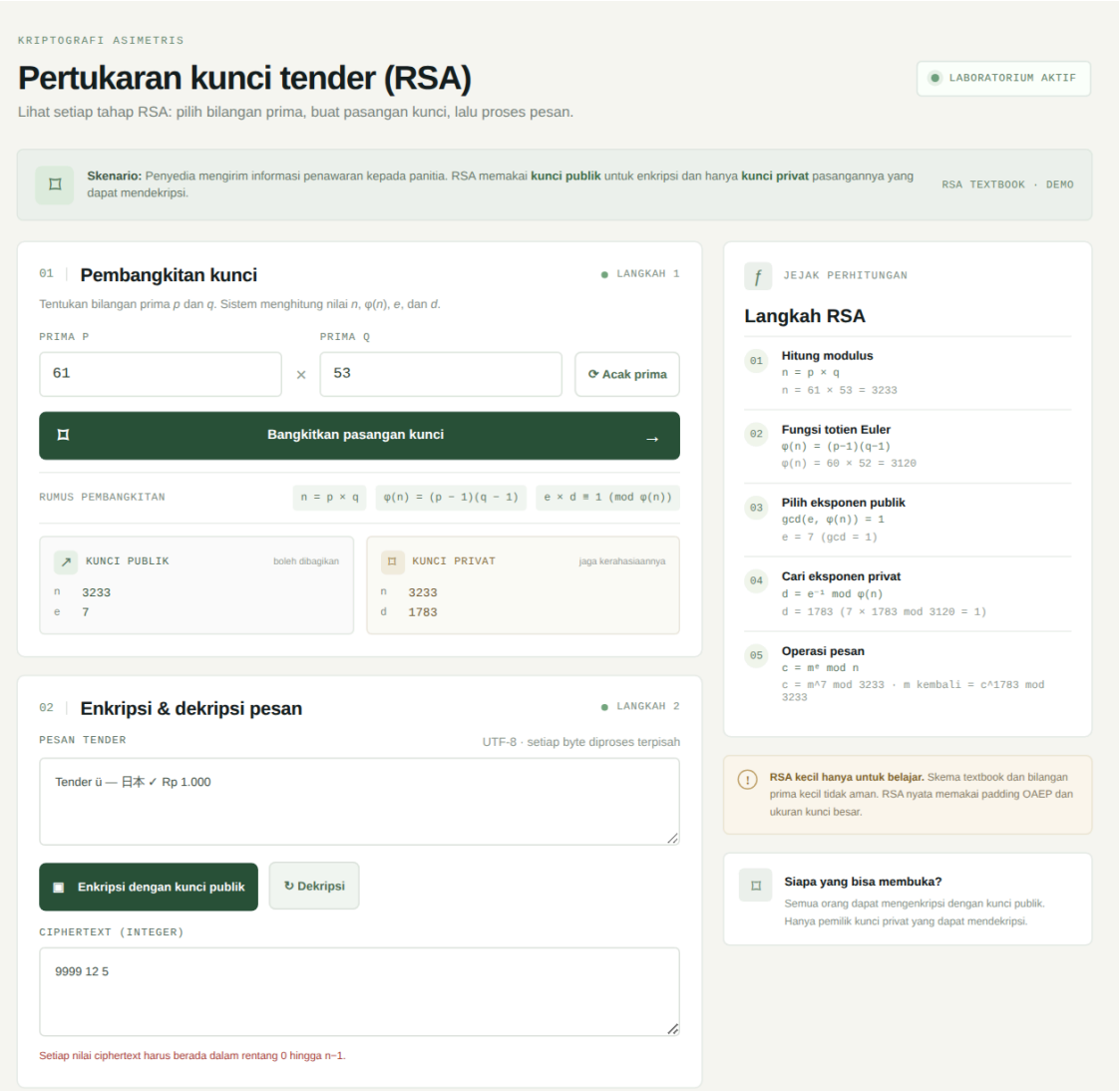
Gambar 4. Prima acak (p = 409, q = 269; n = 110021, e = 65537, d = 30881) dengan pesan Unicode yang berhasil dipulihkan.

Tombol Acak prima memilih dua bilangan prima pada rentang 257–997 lalu membangkitkan kunci baru. Pesan yang memuat karakter non-ASCII (ü, —, 日本, ✓) tetap pulih tepat karena diproses sebagai byte UTF-8.



Gambar 5. Validasi masukan: p = 60 (bukan prima) ditolak dengan pesan galat, kunci tidak dibuat, dan tombol enkripsi/dekripsi nonaktif.

Nilai 60 bukan prima sehingga aplikasi menampilkan pesan galat, mengosongkan kunci dan jejak perhitungan, serta menonaktifkan tombol enkripsi dan dekripsi sampai kunci valid dibangkitkan.



Gambar 6. Ciphertext rusak "9999 12 5" ditolak karena nilai melebihi rentang 0 hingga n−1.

Ciphertext berisi nilai 9999 yang melebihi n − 1 = 3232 sehingga dekripsi ditolak dengan pesan rentang. Aplikasi tetap berjalan dan pesan sebelumnya tidak berubah.

7. Kesimpulan dan Saran

7.1 Kesimpulan

1. TenderVault menyediakan laboratorium RSA berbasis antarmuka web yang mencakup pembangkitan kunci, enkripsi, dan dekripsi pesan teks untuk skenario pertukaran informasi penawaran, seluruhnya tanpa library kriptografi.
2. Fungsi inti RSA (gcd, extendedGcd, modInverse, modPow, isPrime) ditulis sendiri dengan BigInt. Perhitungan kunci bawaan (n = 3233, $\varphi(n)$ = 3120, e = 7, d = 1783) cocok dengan perhitungan manual, dan pesan dapat dipulihkan baik dengan prima bawaan maupun prima acak.
3. Setiap byte pesan UTF-8 dienkripsi terpisah sehingga pesan non-ASCII dapat diproses, dengan syarat $n > 255$ yang divalidasi oleh aplikasi.
4. Dua belas skenario pengujian menunjukkan pesan sampel dapat dipulihkan, masukan tidak valid dan ciphertext rusak ditolak, dan perubahan parameter membatalkan kunci lama. Satu cacat pada ekspresi reguler dekripsi ditemukan dan diperbaiki, dan satu keterbatasan (pesan galat masukan notasi ilmiah) dicatat.
5. Karena ukuran kunci kecil, ketiadaan padding, dan ketiadaan autentikasi, aplikasi hanya sesuai sebagai media pembelajaran dan tidak boleh dipakai untuk data tender yang sebenarnya.

7.2 Saran pengembangan

- Untuk sistem nyata, gunakan library kriptografi yang teruji, RSA-OAEP, dan ukuran kunci yang direkomendasikan [2], [3]; lazimnya RSA melindungi kunci sesi simetris (misalnya kunci AES), bukan dokumen besar.
- Perketat validasi masukan p dan q (bilangan non-integer dan batas ukuran) dan sediakan pesan galat berbahasa Indonesia untuk semua kasus.
- Ganti uji prima pembagian percobaan dengan uji probabilistik seperti Miller–Rabin agar prima yang lebih besar dapat dipakai tanpa memperlambat browser.
- Tambahkan tanda tangan digital untuk autentikasi dan integritas, serta fitur ekspor/impor kunci agar ciphertext lama dapat didekripsi tanpa mengetik ulang p dan q.
- Integrasikan RSA dengan enkripsi simetris (laboratorium DES/AES pada proyek induk) sehingga kunci simetris dibungkus kunci publik RSA, sesuai praktik nyata.
- Perluas pengujian dengan lintas browser, uji fuzzing, dan review oleh pihak lain.

Daftar Pustaka

[1] Rivest, R. L., Shamir, A., dan Adleman, L. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems.” *Communications of the ACM* 21(2), 1978, hlm. 120–126.

[2] Moriarty, K., Kaliski, B., Jonsson, J., dan Rusch, A. *RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2*. IETF, 2016.

[3] National Institute of Standards and Technology (NIST). *SP 800-56B Rev. 2: Recommendation for Pair-Wise Key-Establishment Using Integer Factorization Cryptography.* 2019.

[4] Rosen, K. H. *Discrete Mathematics and Its Applications*, edisi ke-8. McGraw-Hill, 2019. (Algoritma Euclid, invers modular, dan perpangkatan modular.)

[5] Stallings, W. *Cryptography and Network Security: Principles and Practice*, edisi ke-7. Pearson, 2017.